

REDES PROGRAMABLES CON P4

Etapa 3 del Proyecto

Fecha de Entrega: 29 de junio del 2026

A — Caché DNS

B — Anomalías

C — SRv6

D — VXLAN

Curso de Redes de Computadoras

Ciclo Lectivo 2026

1. Descripción General del Proyecto

Cada persona seleccionará una de las cuatro opciones temáticas disponibles e implementará una solución completa usando el lenguaje P4 sobre BMv2/Mininet. Se espera que el proyecto demuestre dominio técnico del plano de datos programable, capacidad de depuración y habilidad para comunicar resultados.

2. Requisitos Generales — Todas las Opciones

2.1 Informe técnico

- Mínimo 5 páginas (sin portada ni referencias).
- Introducción con motivación del tema, estado del arte y casos de uso reales.
- Descripción detallada del diseño: parsers, tablas, pipeline P4.
- Descripción del control plane y cómo se poblan las tablas.
- Resultados experimentales: métricas de rendimiento, capturas de tráfico, análisis.
- Conclusiones y trabajo futuro.
- Mínimo 5 referencias (IEEE, ACM, NSDI, SIGCOMM, p4.org, RFCs).

2.2 Código fuente

- Repositorio Git con historial de commits limpio.
- README.md con instrucciones de instalación y ejecución paso a paso.
- Código P4 comentado y estructurado.
- Scripts de prueba automáticos (Python/Bash) que cubran los casos principales.

2.3 Demostración

- Defensa oral de 10 minutos ante el profesor mostrando la topología activa y los casos de prueba. Debe incluir explicación del pipeline P4 y ejecución de pruebas en vivo.

3. Opciones del Proyecto

Opción A	Caché DNS en el Data Plane
---------------------	-----------------------------------

Descripción

Implementar un sistema de caché DNS directamente dentro del plano de datos P4. El switch interceptará consultas DNS, responderá aquellas cuya respuesta ya tenga cacheada y dejará

pasar al servidor DNS real las consultas nuevas. El objetivo es demostrar la reducción de latencia medible cuando los registros están en caché.

Funcionalidades obligatorias

- Parsing de paquetes DNS sobre UDP (puerto 53); soporte mínimo de registros tipo A.
- Tabla de caché implementada con registros P4: almacenar nombre de dominio (hash) → IP.
- Lógica de hit/miss: respuesta directa en hit, reenvío al servidor DNS real en miss.
- Actualización del caché desde el control plane al recibir respuestas del servidor real.
- Contador de hits y misses legible desde el controlador Python.

Funcionalidades opcionales (+5% c/u)

- Soporte de TTL del registro DNS: invalidar entradas expiradas.
- Soporte de registros AAAA (IPv6).
- Dashboard en consola que muestre hit rate en tiempo real.

Topología requerida

- 1 switch P4, al menos 4 clientes DNS, 1 servidor DNS real (simulado con Scapy o dnsmasq).
- Demostrar latencia con y sin caché (usar time en queries repetidas).

Entregables específicos

Entregable	Descripción / Criterio de aceptación
Código P4 del caché	Parsing correcto de DNS, lógica hit/miss funcionando
Controlador Python	Poblado del caché y lectura de contadores
Prueba de latencia	Gráfica o tabla comparando latencia hit vs. miss vs. sin caché
Análisis de capacidad	Cuántos registros puede almacenar el caché y por qué

Opción B	Detección de Anomalías de Tráfico
-----------------	--

Descripción

Implementar un sistema de detección de anomalías de red dentro del plano de datos P4. El switch analizará el tráfico en tiempo real para identificar comportamientos anómalos como port scans, ataques de fuerza bruta SSH, o floods UDP, tomando acciones de mitigación (rate limiting o drop) sin intervención del control plane.

Funcionalidades obligatorias

- Implementación de Count-Min Sketch (CMS) para estimar frecuencia de flujos.
- Detección de port scan horizontal: misma IP origen, múltiples IPs/puertos destino en ventana de tiempo.
- Detección de flood: flujo individual que supere un umbral de pps configurable.
- Acción de mitigación: marcar paquetes sospechosos (DSCP) o descartarlos.
- Reporte al control plane cuando se detecta anomalía (digest o clone de paquete).

Funcionalidades opcionales (+5% c/u)

- Detección de SYN flood (half-open connections usando registers).
- Lista blanca de IPs inmunes a la detección.
- Visualización en tiempo real de los flujos sospechosos en consola.

Topología requerida

- 1 switch P4, al menos 4 hosts (algunos usados como atacantes simulados con Scapy).
- Demostrar detección correcta con tasa de falsos positivos documentada.
- Demostrar que el tráfico legítimo no se ve afectado durante un ataque.

Entregables específicos

Entregable	Descripción / Criterio de aceptación
Código P4 con CMS	Implementación correcta del sketch y los umbrales
Script de ataque simulado	Genera port scan y flood controlado con Scapy
Script de tráfico legítimo	Tráfico de fondo que no debe verse afectado
Análisis de precisión	Tabla con TP, FP, TN, FN para al menos 3 escenarios

**Opción
C****Segment Routing sobre IPv6 (SRv6)****Descripción**

Implementar un subconjunto funcional de Segment Routing sobre IPv6 (SRv6) en P4. Los estudiantes construirán nodos que puedan actuar como ingress (insertan la cabecera SRH), transit (actualizan el segmento activo) y egress (eliminan la cabecera SRH). El objetivo es demostrar enrutamiento explícito por caminos predefinidos.

Funcionalidades obligatorias

- Parsing del encabezado IPv6 y del Segment Routing Header (SRH, RFC 8754).
- Nodo ingress: encapsula tráfico IPv4/IPv6 original en un paquete SRv6, inserta lista de segmentos.
- Nodo transit: decrementa Segments Left, actualiza IPv6 DA con el próximo segmento.
- Nodo egress: detecta Segments Left = 0, extrae payload original y lo reenvía.
- Control plane Python que configure los segmentos para al menos 2 rutas distintas al mismo destino.

Funcionalidades opcionales (+5% c/u)

- Traffic engineering: selección de ruta según QoS del tráfico (puerto TCP).
- Telemetría: insertar metadatos de latencia en el SRH en cada nodo (INT simplificado).
- Fast Reroute: detectar fallo de un segmento y redirigir a ruta alternativa.

Topología requerida

- Mínimo 4 nodos P4 (router SRv6) formando una topología con al menos 2 caminos posibles entre origen y destino.
- Demostrar que el mismo flujo sigue caminos distintos según la política configurada.
- Capturar y analizar cabeceras SRH en tránsito con Scapy.

Entregables específicos

Entregable	Descripción / Criterio de aceptación
Código P4 (ingress, transit, egress)	Los tres roles implementados correctamente
Controlador Python	Configuración de listas de segmentos para múltiples rutas
Capturas de paquetes	Wireshark/Scapy mostrando el SRH en cada nodo
Comparación de rutas	Demostración de tráfico siguiendo caminos distintos

**Opción
D****Túneles VXLAN en el Data Plane****Descripción**

Implementar el plano de datos de VXLAN (RFC 7348) completamente en P4. Los VTEPs (VXLAN Tunnel End Points) serán switches P4 que encapsulen y desencapsulen tráfico Ethernet en UDP/IP, permitiendo extender redes L2 sobre una red IP subyacente (underlay), como ocurre en datacenters modernos.

Funcionalidades obligatorias

- Parsing de Ethernet / IP / UDP / VXLAN header (8 bytes) / inner Ethernet.
- Lógica de encapsulación (VTEP origen): lookup de MAC destino → VNI → IP del VTEP remoto, encapsula en VXLAN/UDP/IP/Ethernet outer.
- Lógica de desencapsulación (VTEP destino): strip outer headers, reenvío del inner frame a la red local.
- Tabla de MAC learning en el control plane: el controlador aprende qué MACs están detrás de cada VTEP.
- Soporte de al menos 2 VNIs (segmentos) distintos activos simultáneamente.

Funcionalidades opcionales (+5% c/u)

- BUM (Broadcast, Unknown, Multicast) handling: flooding controlado a todos los VTEPs del mismo VNI.
- VTEP discovery automático: aprender nuevos VTEPs dinámicamente al recibir tráfico encapsulado.
- Soporte de VXLAN-GPE para llevar metadatos adicionales (INT).

Topología requerida

- Mínimo 2 VTEPs P4 interconectados por una red IP underlay (puede ser simple).
- Al menos 2 VNIs activos, cada uno con 2 hosts, demostrando aislamiento entre VNIs.
- Demostrar conectividad L2 entre hosts del mismo VNI en distintos VTEPs.
- Demostrar que hosts de VNIs distintos no pueden comunicarse directamente.

Entregables específicos

Entregable	Descripción / Criterio de aceptación
Código P4 (encap + decap)	Encapsulación y desencapsulación VXLAN correctas
Controlador Python con MAC table	Aprendizaje y reenvío basado en VNI + MAC

Prueba de aislamiento	Demostrar que VNI A y VNI B están aislados
Captura de tráfico	Mostrar outer e inner headers en Wireshark/Scapy

4. Criterios de Evaluación

La nota del proyecto se compone de criterios transversales (aplicables a todas las opciones) y criterios específicos de funcionalidad según la opción elegida.

4.1 Criterios transversales (60%)

Criterio	%	Indicadores
Informe técnico	20%	Profundidad, estado del arte, análisis de resultados, redacción
Calidad del código P4	10%	Modularidad, comentarios, uso correcto de primitivas P4
Controlador y scripts de prueba	10%	Funcionalidad, cobertura de casos, instrucciones claras
Defensa oral	20%	Comprensión del diseño, respuesta a preguntas técnicas

4.2 Criterios de funcionalidad por opción (40%)

Opción	Aspecto evaluado	%	Indicadores clave
A — DNS	Correctitud del caché	15%	Hit/miss funciona, respuestas DNS válidas
A — DNS	Métricas de latencia	15%	Reducción medible y documentada de latencia
B — Anomalías	Precisión de detección	20%	Alta tasa TP, baja tasa FP documentada
B — Anomalías	Mitigación efectiva	10%	Tráfico legítimo no afectado
C — SRv6	Correctitud SRH	20%	Parsing y manipulación del SRH según RFC 8754
C — SRv6	Enrutamiento por segmentos	10%	Dos rutas distintas demostrables
D — VXLAN	Encap/decap correcta	20%	Frames internos entregados correctamente

D — VXLAN	Aislamiento por VNI	10%	VNIs distintos no se comunican
----------------------	---------------------	------------	--------------------------------

Nota: las funcionalidades opcionales de cada opción pueden sumar hasta un 10% adicional sobre la nota base de funcionalidad, con un máximo de 100 puntos totales en el proyecto.

5. Normas y Políticas

- Todo el código debe ser original del grupo. El uso de código de terceros debe citarse explícitamente y no puede representar más del 20% del código total.
- El uso de IA generativa para escribir código debe declararse en el README indicando qué partes se generaron y cómo fueron verificadas.
- Debido a condiciones extraordinarias, si la entrega no se hace el día concreto, obtendrá un 0 como nota.
- La no presentación de la defensa oral implica una nota máxima de 70 puntos.

6. Recursos Recomendados

- P4 Language Specification — <https://p4.org/p4-spec/>
- p4lang/tutorials — <https://github.com/p4lang/tutorials>
- RFC 7348 (VXLAN) — <https://datatracker.ietf.org/doc/html/rfc7348>
- RFC 8754 (SRv6 Network Programming) — <https://datatracker.ietf.org/doc/html/rfc8754>
- CMS (Count-Min Sketch) — Cormode & Muthukrishnan, 2005.
- NetCache: Balancing Key-Value Stores with Fast In-Network Caching — SOSP 2017.
- Hula et al., Real-time DDoS Mitigation in Programmable Data Planes — SIGCOMM 2020.